

Singleton Design Pattern — Bilingual Study Document (English + Hindi)

Inspired by algomaster.io/learn/lld/singleton — concepts and structure preserved, written in original wording with self-authored PHP examples

Study Companion · Updated July 2026

Singleton Design Pattern

Note on sourcing / स्रोत पर टिप्पणी: AlgoMaster.io is a paid interview-prep course, and its code samples sit behind a subscription (its Singleton lesson's implementation sections show section headers with the actual code stripped out in a plain fetch). This document follows the same topic order and teaching structure as their Singleton lesson — the single-instance/global-access framing, a print-spooler analogy, the class diagram, the request-creation workflow, five implementation variants from naive to production-grade, a cache-manager worked example, then pros and cons — but every explanation below is written fresh in my own words, and all PHP code is original.

AlgoMaster.io एक पेड (paid) इंटरव्यू-प्रेप कोर्स है, और इसके कोड सैंपल सब्सक्रिप्शन (subscription) के पीछे हैं (इसके Singleton पाठ के इम्प्लीमेंटेशन सेक्शन्स में सिर्फ हेडिंग्स दिखीं, असली कोड गायब था)। यह दस्तावेज़ उनके Singleton पाठ जैसा ही विषय-क्रम और टीचिंग स्ट्रक्चर फ़ॉलो करता है — सिंगल-इंस्टेंस/ग्लोबल-एक्सेस की फ्रेमिंग, एक प्रिंट-स्पूलर उपमा, क्लास डायग्राम, रिक्वेस्ट/क्रिएशन वर्कफ़्लो, नैव से लेकर प्रोडक्शन-ग्रेड तक पाँच इम्प्लीमेंटेशन वैरिएंट्स, एक cache-manager वर्किंग उदाहरण, फिर फ़ायदे और नुक़सान — लेकिन नीचे दी गई हर व्याख्या मेरे अपने शब्दों में ताज़ा लिखी गई है, और सारा PHP कोड ओरिजिनल है।

Overview

In real systems, some classes genuinely should only ever have one live object — a thread pool, a shared cache, an application logger. Letting more than one exist isn't just wasteful; it can produce outright incorrect behavior, wasted resources, or results that silently contradict each other depending on which copy answered the call. The Singleton pattern exists specifically to close off that possibility.

असली सिस्टम्स में, कुछ क्लासेज़ ऐसी होती हैं जिनका वाकई सिर्फ़ एक ही जीवंत (live) ऑब्जेक्ट होना चाहिए — एक थ्रेड पूल (thread pool), एक साझा कैश (shared cache), एक ऐप्लिकेशन लॉगर। एक से ज़्यादा को मौजूद रहने देना सिर्फ़ फ़िज़ूलखर्ची नहीं है — यह सीधे तौर पर ग़लत व्यवहार, रिसोर्सेज़ की बर्बादी, या ऐसे नतीजे पैदा कर सकता है जो चुपचाप एक-दूसरे से टकराते हैं, इस पर निर्भर करते हुए कि किस कॉपी ने जवाब दिया। Singleton पैटर्न खासतौर पर इसी संभावना को बंद करने के लिए मौजूद है।

It's a deceptively small pattern to describe, and a surprisingly easy one to get subtly wrong in practice — worth taking seriously precisely because it looks trivial at first glance.

इसे बताना बहुत छोटा (deceptively small) पैटर्न लगता है, लेकिन असल इस्तेमाल में इसे सूक्ष्म (subtle) रूप से ग़लत करना आश्चर्यजनक रूप से आसान है — इसे गंभीरता से लेना ज़रूरी है, ठीक इसलिए क्योंकि पहली नज़र में यह मामूली लगता है।

1. What Is the Singleton Pattern?

Definition: Singleton is a creational design pattern that guarantees a class has exactly one instance, and provides one globally reachable point of access to it.

परिभाषा: Singleton एक क्रिएशनल डिज़ाइन पैटर्न है, जो यह गारंटी देता है कि किसी क्लास का बिल्कुल एक ही instance मौजूद हो, और उस तक पहुँचने का एक वैश्विक रूप से (globally) सुलभ स्थान देता है।

Two separate requirements sit inside that one sentence: **exactly one instance** — no matter how many times or from how many places the code asks for it, the same object comes back every time; and **reachable from anywhere** — any part of the codebase can get to that instance without it having to be threaded through constructor parameters or method arguments all the way down the call stack.

उस एक वाक्य के अंदर दो अलग-अलग ज़रूरतें छुपी हैं: **बिल्कुल एक instance** — चाहे कोड कितनी भी बार या कितनी भी जगहों से माँग करे, हर बार वही एक ऑब्जेक्ट वापस मिलता है; और **कहीं से भी पहुँच योग्य** — कोडबेस का कोई भी हिस्सा उस instance तक पहुँच सकता है, बिना उसे कंस्ट्रक्टर पैरामीटर्स या मेथड आर्गुमेंट्स के ज़रिए पूरे कॉल स्टैक (call stack) में पिरोए।

A real-world analogy: think of a single print spooler managing every print job for a whole office, rather than each application maintaining its own. If every application spun up its own spooler, jobs could genuinely interleave on the physical printer — pages from two different documents mixed together mid-print. The one shared spooler is what keeps every job coordinated and in order.

एक असल-दुनिया की उपमा: एक ऐसे प्रिंट स्पूलर (print spooler) की कल्पना कीजिए, जो पूरे ऑफ़िस के हर प्रिंट जॉब को संभालता है, बजाय इसके कि हर ऐप्लिकेशन अपना खुद का स्पूलर रखे। अगर हर ऐप्लिकेशन अपना स्पूलर बना ले, तो असली प्रिंटर पर जॉब्स सच में आपस में मिल सकते हैं — दो अलग-अलग दस्तावेज़ों के पन्ने बीच में मिल जाएँ। वही एक साझा स्पूलर है जो हर जॉब को समन्वित (coordinated) और क्रम में रखता है।

Where this shape of problem shows up in practice: managing a shared resource that shouldn't be duplicated (a database connection pool, a thread pool, an application-wide configuration object); coordinating something system-wide (logging output, a print spooler, a central file-manager abstraction); or holding state that must stay singular and consistent (a user session context, overall application state).

यह किस्म की समस्या असल में कहाँ दिखती है: एक साझा रिसोर्स को मैनेज करना जिसे डुप्लीकेट नहीं होना चाहिए (डेटाबेस कनेक्शन पूल, थ्रेड पूल, एक ऐप-वाइड कॉन्फ़िगरेशन ऑब्जेक्ट); कुछ ऐसा जो पूरे सिस्टम में समन्वित होना चाहिए (लॉगिंग आउटपुट, एक प्रिंट स्पूलर, एक केंद्रीय फ़ाइल-मैनेजर एब्स्ट्रैक्शन); या ऐसी स्टेट रखना जो हमेशा एकवचन (singular) और संगत (consistent) रहनी चाहिए (एक यूज़र सेशन कॉन्टेक्स्ट, पूरे ऐप्लिकेशन की स्टेट)।

2. Class Diagram

To make "exactly one instance" actually enforceable, two things have to be true structurally: outside code must be unable to construct the class directly, and the class must expose exactly one sanctioned way to reach its instance.

"बिल्कुल एक instance" को असल में लागू करने योग्य (enforceable) बनाने के लिए, संरचनात्मक (structurally) रूप से दो बातें सच होनी चाहिए: बाहरी कोड सीधे क्लास को कंस्ट्रक्ट न कर पाए, और क्लास अपने instance तक पहुँचने का बिल्कुल एक ही स्वीकृत (sanctioned) तरीका एक्सपोज़ करे।

Structurally: a private static field holds the one instance; the constructor is private (or otherwise restricted) so nothing outside the class can call it directly; and a public static accessor method returns the shared instance to anyone who asks.

संरचना के अनुसार: एक प्राइवेट स्टैटिक फ़ील्ड उस एक instance को रखती है; कंस्ट्रक्टर प्राइवेट (या किसी और तरह प्रतिबंधित) होता है, ताकि क्लास के बाहर कोई भी उसे सीधे कॉल न कर सके और एक पब्लिक स्टैटिक एक्सेसर (accessor) मेथड जो भी माँगे उसे साझा instance लौटा देती है।

Why not just use a plain global variable instead? A language-level global gives you similar reachability, but none of the control — nothing stops it from being created eagerly whether it's needed or not, nothing enforces that exactly one ever exists, and there's no natural place to add thread-safety at construction time. A Singleton keeps all of that control inside one class: it decides exactly when and how its one instance comes into being, it can defer that work until first use, and it can guard that first-use moment against concurrent callers.

सिर्फ़ एक सामान्य ग्लोबल वैरिएबल का उपयोग क्यों नहीं? भाषा-स्तर (language-level) का एक ग्लोबल आपको जैसी पहुँच तो देता है, लेकिन उतना नियंत्रण (control) नहीं — कुछ भी उसे ज़रूरत हो या न हो, तुरंत बनने से नहीं रोकता, कुछ भी यह लागू नहीं करता कि बिल्कुल एक ही मौजूद रहे, और कंस्ट्रक्शन के समय थ्रेड-सेफ़्टी जोड़ने की कोई स्वाभाविक जगह नहीं होती। एक Singleton यह सारा नियंत्रण एक ही क्लास के अंदर रखता है: यह तय करता है कि उसका एक instance ठीक कब और कैसे बने, वह इस काम को पहली ज़रूरत तक टाल सकता है, और उस पहले-इस्तेमाल के पल को समवर्ती (concurrent) कॉलर्स से बचा सकता है।

3. How It Works

The workflow has four steps. **First request:** a caller invokes the static accessor for the first time; it checks whether the shared instance already exists. **Instance creation:** finding none yet, the accessor builds one via the private constructor and stores it in the static field. **Return the instance:** that freshly built object is handed back to the caller. **Every later request:** the accessor finds the instance already sitting in the static field and returns it immediately, skipping construction entirely — two different callers asking at two different times both end up holding a reference to the exact same object.

इस वर्कफ़्लो के चार चरण हैं। **पहली रिक्वेस्ट:** एक कॉलर पहली बार स्टैटिक एक्सेसर को कॉल करता है; यह जाँचता है कि साझा instance पहले से मौजूद है या नहीं। **instance बनाना:** अभी तक कोई न मिलने पर, एक्सेसर प्राइवेट कंस्ट्रक्टर के ज़रिए एक बनाता है और उसे स्टैटिक फ़ील्ड में स्टोर करता है। **instance लौटाना:** वह अभी-अभी बना ऑब्जेक्ट कॉलर को वापस दे दिया जाता है। **हर बाद की रिक्वेस्ट:** एक्सेसर को instance पहले से ही स्टैटिक फ़ील्ड में मिलता है और वह उसे तुरंत लौटा देता है, कंस्ट्रक्शन को पूरी तरह छोड़कर — दो अलग-अलग कॉलर्स, दो अलग-अलग समय पर माँगने के बावजूद, दोनों के पास बिल्कुल उसी एक ऑब्जेक्ट का रेफ़रेंस होता है।

4. Implementation Variants

The central challenge across every variant is the same: **if two threads (or, in PHP terms, two concurrent execution contexts within one long-running process) both call the accessor at nearly the same instant, and neither has finished creating the instance yet, both can end up creating one — silently defeating the entire pattern.** The variants below progress from naive to production-grade, each fixing a specific weakness in the one before it. Code shown is original, written for this document.

हर वैरिएंट में केंद्रीय चुनौती एक जैसी है: अगर दो थ्रेड्स (या, PHP की भाषा में, एक लंबे समय तक चलने वाली प्रोसेस के भीतर दो समवर्ती एक्जीक्यूशन कॉन्टेक्स्ट्स) लगभग एक ही पल में एक्सेसर को कॉल करें, और दोनों में से किसी ने अभी तक **instance** बनाना पूरा न किया हो, तो दोनों एक-एक **instance** बना सकते हैं — जो चुपचाप पूरे पैटर्न को बेकार कर देता है। नीचे दिए वैरिएंट्स नैव (naive) से प्रोडक्शन-ग्रेड (production-grade) तक बढ़ते हैं, हर एक पिछले वैरिएंट की एक खास कमज़ोरी ठीक करते हुए। दिखाया गया कोड ओरिजिनल है, इसी दस्तावेज़ के लिए लिखा गया।

4.1 — Lazy Initialization (Not Thread-Safe)

Builds the instance only on first actual use, saving the cost entirely if it's never needed — but the check-then-create sequence is unguarded, so two concurrent callers can both see "not yet created" and both proceed to build one.

instance की ज़रूरत सिर्फ़ पहली बार असल इस्तेमाल पर बनता है, अगर कभी ज़रूरत ही न पड़े तो पूरी लागत बच जाती है — लेकिन चेक-फिर-बनाओ (check-then-create) क्रम असुरक्षित (unguarded) है, इसलिए दो समवर्ती कॉलर्स दोनों "अभी तक नहीं बना" देख सकते हैं और दोनों एक-एक बनाने लगते हैं।

```
<?php

class LazySingleton
{
    private static ?LazySingleton $instance = null;

    private function __construct()
    {
    }

    // NOT safe under a shared, concurrent process (e.g. Swoole) — two
    // callers can both pass this null-check before either finishes
    // assigning self::$instance.
    public static function getInstance(): self
    {
        if (self::$instance === null) {
            self::$instance = new self();
        }
        return self::$instance;
    }
}
```

4.2 — Thread-Safe (Locked) Singleton

Wraps the whole check-and-create sequence in a lock, so only one caller at a time can be inside it — correct, but every single call pays the cost of acquiring the lock, even long after the instance already exists and there's nothing left to protect.

पूरे चेक-और-बनाओ (check-and-create) क्रम को एक लॉक (lock) में लपेटा जाता है, ताकि एक समय पर सिर्फ़ एक कॉलर ही उसके अंदर हो — सही, लेकिन हर एक कॉल लॉक हासिल करने की लागत चुकाती है, भले ही instance बहुत पहले से मौजूद हो और अब बचाने के लिए कुछ बचा ही न हो।

```
<?php

final class LockedSingleton
{
    private static ?LockedSingleton $instance = null;
    private static ?\Swoole\Lock $lock = null;

    private function __construct()
    {
    }

    public static function getInstance(): self
    {
        self::$lock ??= new \Swoole\Lock(SWOOLE_MUTEX);
        self::$lock->lock(); // every call pays this cost, even call #10,000
        try {
            if (self::$instance === null) {
                self::$instance = new self();
            }
            return self::$instance;
        } finally {
            self::$lock->unlock();
        }
    }
}
```

4.3 — Double-Checked Locking

Only takes the lock on the (rare) path where the instance might not exist yet; once it exists, a lock-free check at the top short-circuits every future call, so the locking cost is paid a handful of times total, not on every call.

लॉक सिर्फ़ उस (दुर्लभ) रास्ते पर लिया जाता है जहाँ instance शायद अभी तक न बना हो; एक बार बन जाने पर, ऊपर एक बिना-लॉक (lock-free) चेक हर आगे की कॉल को तुरंत निपटा देता है, इसलिए लॉकिंग की लागत कुल मिलाकर बस कुछ ही बार चुकानी पड़ती है, हर कॉल पर नहीं।

```

<?php

final class DoubleCheckedSingleton
{
    private static ?DoubleCheckedSingleton $instance = null;
    private static ?\Swoole\Lock $lock = null;

    private function __construct()
    {
    }

    public static function getInstance(): self
    {
        if (self::$instance !== null) {
            return self::$instance; // fast path – no locking after warm-up
        }

        self::$lock ??= new \Swoole\Lock(SWOOLE_MUTEX);
        self::$lock->lock();
        try {
            if (self::$instance === null) { // re-check: another caller may have won
the race
                self::$instance = new self();
            }
        } finally {
            self::$lock->unlock();
        }
        return self::$instance;
    }
}

```

Note specific to PHP, not present in most other languages' write-ups of this pattern: Java's version of double-checked locking additionally requires the instance field to be marked `volatile`, because the JVM's memory model otherwise permits instruction reordering that can expose a partially-constructed object to another thread. PHP has no `volatile` keyword or equivalent memory-visibility primitive — the practical PHP answer is either the `Swoole\Lock`-guarded version above, under a genuinely shared-process deployment, or simply not needing this at all under classic PHP-FPM, where each request already runs in total isolation from every other.

यह टिप्पणी खासतौर पर *PHP* के लिए है, इस पैटर्न के ज़्यादातर दूसरी भाषाओं के विवरण में मौजूद नहीं होती: Java के double-checked locking वर्शन में instance फ़ील्ड को अतिरिक्त रूप से `volatile` चिह्नित करना ज़रूरी है, क्योंकि JVM का मेमोरी मॉडल अन्यथा ऐसी इंस्ट्रक्शन रीऑर्डिंग (reordering) की अनुमति देता है, जो किसी और थ्रेड को आंशिक रूप से बना (partially-constructed) ऑब्जेक्ट दिखा सकती है। PHP में कोई `volatile` कीवर्ड या बराबर की मेमोरी-विज़िबिलिटी प्रिमिटिव नहीं है — व्यावहारिक PHP जवाब है या तो ऊपर दिया गया `Swoole\Lock`-गार्डेड वर्शन, एक वाकई साझा-प्रोसेस डिप्लॉयमेंट के तहत, या क्लासिक PHP-FPM के तहत इसकी ज़रूरत ही न होना, जहाँ हर रिक्वेस्ट पहले से ही हर दूसरी रिक्वेस्ट से पूरी तरह अलग-थलग चलती है।

4.4 — Eager Initialization

Builds the instance immediately, before any concurrent access is even possible — inherently safe with zero locking, at the cost of always paying the construction cost even if the instance is never actually used on a given run.

instance को तुरंत बनाया जाता है, इससे पहले कि कोई समवर्ती एक्सेस संभव भी हो — बिना किसी लॉकिंग के स्वाभाविक रूप से (inherently) सुरक्षित, इस कीमत पर कि कंस्ट्रक्शन की लागत हमेशा चुकानी पड़ती है, भले ही किसी खास रन में instance का इस्तेमाल कभी हो ही नहीं।

```
<?php

final class EagerSingleton
{
    // Created the moment this file/class is loaded – no lazy check needed,
    // and therefore no concurrency race is even possible.
    private static EagerSingleton $instance;

    private function __construct()
    {
    }

    public static function boot(): void
    {
        self::$instance = new self();
    }

    public static function getInstance(): self
    {
        return self::$instance;
    }
}

EagerSingleton::boot(); // called once, at application/worker startup
```

4.5 — A PHP-Ecosystem Equivalent of "Language-Specific, Recommended" Approaches

Other languages have a runtime-guaranteed idiom that sidesteps manual locking entirely — Java's enum-based singleton (instantiation guaranteed exactly-once by the JVM's classloader) is the best-known example. PHP has no exact equivalent, but the closest practical parallel in a modern PHP codebase is **not implementing the classic pattern by hand at all** — registering the class as a singleton binding in a DI container (Laravel's `app()->singleton()`, for instance) and letting the container's own resolution-caching logic guarantee "resolved once, reused after," the same way a JVM enum's classloader guarantees single-instantiation, without the class itself needing a private constructor or a hand-rolled lock.

दूसरी भाषाओं में एक रनटाइम-गारंटीड (runtime-guaranteed) मुहावरा (idiom) होता है जो मैनुअल लॉकिंग को पूरी तरह टाल देता है — Java का enum-आधारित सिंगलटन (JVM के क्लासलोडर द्वारा गारंटीड बिल्कुल-एक-बार इंस्टैंशिएशन) इसका सबसे जाना-

पहचाना उदाहरण है। PHP में इसका ठीक बराबर कुछ नहीं है, लेकिन एक आधुनिक PHP कोडबेस में सबसे नज़दीकी व्यावहारिक समानांतर है **क्लासिक पैटर्न को हाथ से बिल्कुल भी इम्प्लीमेंट न करना** — क्लास को एक DI कंटेनर में सिंगलटन बाइंडिंग (जैसे Laravel का `app()->singleton()`) के रूप में रजिस्टर करना, और कंटेनर के अपने रिज़ॉल्यूशन-कैशिंग (resolution-caching) लॉजिक को "एक बार रिज़ॉल्व, बाद में दोबारा इस्तेमाल" की गारंटी देने देना — बिल्कुल वैसे ही जैसे एक JVM enum का क्लासलोडर सिंगल-इंस्टैंशिएशन की गारंटी देता है, बिना क्लास को खुद एक प्राइवेट कंस्ट्रक्टर या हाथ से बनाए लॉक की ज़रूरत के।

5. Practical Example: In-Memory Cache Manager

Picture an application where several unrelated components — HTTP request handlers, the database layer, background jobs — all need to cache the same kind of expensive-to-fetch data (user profiles, resolved configuration, query results). The goal is one shared cache, so that a write from any one component is immediately visible to every other component, with no duplicate maps quietly drifting out of sync and no memory wasted holding the same data multiple times over.

एक ऐसे ऐप्लिकेशन की कल्पना कीजिए जहाँ कई असंबंधित (unrelated) कॉम्पोनेंट्स — HTTP रिक्वेस्ट हैंडलर्स, डेटाबेस लेयर, बैकग्राउंड जॉब्स — सभी को एक जैसा महँगा-सा (fetch करने में) डेटा कैश करना है (यूज़र प्रोफ़ाइल्स, रिज़ॉल्व्ड कॉन्फ़िगरेशन, क्वेरी नतीजे)। लक्ष्य है एक साझा कैश, ताकि किसी भी एक कॉम्पोनेंट का लिखा हुआ (write) डेटा तुरंत बाक़ी सभी कॉम्पोनेंट्स को दिखे, बिना डुप्लीकेट मैप्स के चुपचाप अलग-अलग हो जाने के, और बिना एक ही डेटा को कई बार मेमोरी में रखने की बर्बादी के।

Without a Singleton, each component would likely instantiate its own cache map, and the moment two of them cache the same key with different values, there's no way to say which one is "correct" — they've silently diverged. With a Singleton `CacheManager`, every component reaches the same shared store, TTL-based expiry can be handled by reading through one code path, and access can be synchronized internally in exactly one place instead of being the caller's problem to solve repeatedly.

Singleton के बिना, हर कॉम्पोनेंट संभवतः अपना खुद का कैश मैप बनाएगा, और जैसे ही दो कॉम्पोनेंट्स एक ही key को अलग-अलग वैल्यूज़ के साथ कैश करें, यह कहने का कोई तरीक़ा नहीं बचता कि कौन-सा "सही" है — वे चुपचाप अलग-अलग हो चुके हैं। एक Singleton `CacheManager` के साथ, हर कॉम्पोनेंट उसी साझा स्टोर तक पहुँचता है, TTL-आधारित एक्सपायरी को एक ही कोड-पथ से पढ़कर संभाला जा सकता है, और एक्सेस को अंदर से बिल्कुल एक ही जगह पर सिंक्रोनाइज़ (synchronized) किया जा सकता है, बजाय इसके कि यह हर कॉलर की बार-बार हल करने वाली समस्या बने।

```

<?php

final class CacheManager
{
    private static ?CacheManager $instance = null;

    /** @var array<string, array{value: mixed, expiresAt: int}> */
    private array $store = [];

    private function __construct()
    {
    }

    public static function getInstance(): self
    {
        if (self::$instance === null) {
            self::$instance = new self();
        }
        return self::$instance;
    }

    public function put(string $key, mixed $value, int $ttlSeconds): void
    {
        $this->store[$key] = [
            'value' => $value,
            'expiresAt' => time() + $ttlSeconds,
        ];
    }

    public function get(string $key): mixed
    {
        if (!isset($this->store[$key])) {
            return null;
        }

        // Lazy TTL cleanup: expired entries are only actually removed the
        // next time someone reads that specific key, rather than via a
        // background sweep – a common, low-overhead real-world choice.
        if ($this->store[$key]['expiresAt'] < time()) {
            unset($this->store[$key]);
            return null;
        }

        return $this->store[$key]['value'];
    }

    public function __clone()
    {
        throw new \LogicException('Cloning this Singleton is not allowed.');
```

```

    }

    // Two unrelated "components" sharing the same cache:
    // दो असंबंधित "कॉम्पोनेंट्स" एक ही कैश को साझा कर रहे हैं:

    function httpHandlerReadsProfile(string $userId): void
    {
        $cache = CacheManager::getInstance();
```

```

    $profile = $cache->get("profile:{$userId}");
    if ($profile === null) {
        $profile = ['id' => $userId, 'name' => 'Fetched From DB'];
        $cache->put("profile:{$userId}", $profile, ttlSeconds: 60);
    }
    echo "HTTP handler sees: " . json_encode($profile) . "\n";
}

function backgroundJobUpdatesProfile(string $userId): void
{
    $cache = CacheManager::getInstance();
    $cache->put("profile:{$userId}", ['id' => $userId, 'name' => 'Updated By Background Job'], ttlSeconds: 60);
}

httpHandlerReadsProfile('u1');
backgroundJobUpdatesProfile('u1');
httpHandlerReadsProfile('u1'); // sees the background job's update immediately

```

Expected output / अपेक्षित आउटपुट:

```

HTTP handler sees: {"id":"u1","name":"Fetched From DB"}
HTTP handler sees: {"id":"u1","name":"Updated By Background Job"}

```

Benefits this achieves: one shared cache with no duplicate storage; any write is immediately visible everywhere; expiry logic lives in exactly one place; and no component needs a cache reference threaded through its constructor — it just asks the `CacheManager` for itself.

इससे मिलने वाले फ़ायदे: एक साझा कैश, कोई डुप्लीकेट स्टोरेज नहीं; कोई भी write हर जगह तुरंत दिखता है; एक्सपायरी लॉजिक बिल्कुल एक ही जगह रहता है; और किसी भी कॉम्पोनेंट को अपने कंस्ट्रक्टर में कैश रेफरेंस पिरोने की ज़रूरत नहीं — वह बस खुद `CacheManager` से माँग लेता है।

6. Pros and Cons

Pros:

- Guarantees exactly one instance and gives every part of the codebase one place to reach it from.
- For a resource-heavy class, paying the construction cost exactly once instead of repeatedly is a genuine, measurable win.
- Naturally supports lazy loading — the instance is only built when it's genuinely first needed.
- Every consumer is guaranteed to be working against the same shared state, with no risk of silently divergent copies.

फ़ायदे:

- बिल्कुल एक instance की गारंटी देता है और कोडबेस के हर हिस्से को उस तक पहुँचने की एक जगह देता है।
- एक रिसोर्स-हैवी (resource-heavy) क्लास के लिए, कंस्ट्रक्शन की लागत बार-बार चुकाने के बजाय बिल्कुल एक बार चुकाना, एक असली और मापने योग्य (measurable) फ़ायदा है।
- स्वाभाविक रूप से लेज़ी लोडिंग (lazy loading) को सपोर्ट करता है — instance सिर्फ़ तभी बनता है जब वाकई पहली बार ज़रूरत हो।
- हर उपभोक्ता (consumer) की गारंटी होती है कि वह उसी साझा स्टेट पर काम कर रहा है, बिना चुपचाप अलग हो चुकी कॉपीज़ के जोखिम के।

Cons:

- Bundles two separate responsibilities into one class — controlling instance count, and providing global access — a direct Single Responsibility Principle tension.
- Needs deliberate, careful handling in any environment with real concurrency, or race conditions during first creation become a real risk.
- Introduces genuine global state into the application, which tends to be harder to reason about as a codebase grows.
- Classes that lean on a Singleton tend to become tightly coupled to it, since reaching it is so convenient that the coupling often isn't even noticed as it accumulates.
- Makes unit testing meaningfully harder, precisely because of the global state it introduces — swapping in a test double isn't as simple as it would be for a normally constructed, injected dependency.

नुक़सान:

- दो अलग-अलग ज़िम्मेदारियों को एक ही क्लास में बाँध देता है — instance की संख्या नियंत्रित करना, और ग्लोबल एक्सेस देना — यह सीधे तौर पर सिंगल रिस्पॉन्सिबिलिटी प्रिंसिपल से टकराता है।

- असली कंकरेंसी (concurrency) वाले किसी भी माहौल में जान-बूझकर, सावधानीपूर्वक हैंडलिंग चाहिए, वरना पहली बार instance बनते समय रेस कंडीशन (race condition) का असली जोखिम बन जाता है।
- ऐप्लिकेशन में असली ग्लोबल स्टेट लाता है, जिसे कोडबेस बढ़ने के साथ समझना मुश्किल होता जाता है।
- जो क्लासेज़ किसी Singleton पर निर्भर होती हैं, वे अक्सर उससे कसकर जुड़ (tightly coupled) जाती हैं, क्योंकि उस तक पहुँचना इतना सुविधाजनक होता है कि यह जुड़ाव अक्सर बढ़ते हुए नज़र ही नहीं आता।
- यूनिट टेस्टिंग को काफ़ी मुश्किल बना देता है, ठीक इसलिए क्योंकि यह ग्लोबल स्टेट लाता है — एक टेस्ट डबल (double) से बदलना उतना आसान नहीं रहता जितना एक सामान्य रूप से बनाई और इंजेक्ट की गई डिपेंडेंसी के लिए होता।

Used carefully, in genuinely singular scenarios, this pattern earns its place. Reached for out of habit, it quietly makes a codebase harder to test and reason about — worth weighing dependency injection as the alternative before defaulting to it, exactly as the source material itself concludes.

सावधानी से, वाक़ई एकवचन (singular) परिस्थितियों में इस्तेमाल किया जाए, तो यह पैटर्न अपनी जगह खुद बनाता है। आदतन (out of habit) इस्तेमाल किया जाए, तो यह चुपचाप कोडबेस को टेस्ट करना और समझना मुश्किल बना देता है — डिफ़ॉल्ट रूप से इसे अपनाने से पहले डिपेंडेंसी इंजेक्शन (dependency injection) को एक विकल्प के तौर पर तौलना उचित है, ठीक वैसे ही जैसे स्रोत सामग्री खुद भी निष्कर्ष निकालती है।

Technical Words Glossary / तकनीकी शब्दों की शब्दावली

English Term	Hindi Translation / हिंदी अनुवाद	Example / उदाहरण
Creational Pattern	क्रिएशनल पैटर्न	Singleton, Factory, और Builder — तीनों क्रिएशनल पैटर्न हैं।
Instance	इंस्टेंस	<code>CacheManager::getInstance()</code> हमेशा एक ही instance लौटाता है।
Global Access Point	ग्लोबल एक्सेस पॉइंट	<code>CacheManager</code> को कोडबेस में कहीं से भी <code>getInstance()</code> से एक्सेस किया जा सकता है।
Thread-Safety	थ्रेड-सेफ्टी	<code>LockedSingleton</code> लॉक का उपयोग करके थ्रेड-सेफ्टी सुनिश्चित करता है।
Race Condition	रेस कंडीशन	दो कॉलर्स के एक साथ instance बनाने की कोशिश करने से रेस कंडीशन बनती है।
Lazy Initialization	लेज़ी इनिशियलाइज़ेशन	<code>LazySingleton</code> में instance सिर्फ़ पहली <code>getInstance()</code> कॉल पर बनता है।
Eager Initialization	ईगर इनिशियलाइज़ेशन	<code>EagerSingleton</code> में instance क्लास लोड होते ही बन जाता है।
Double-Checked Locking	डबल-चेकड लॉकिंग	लॉक लेने से पहले और बाद में, दोनों बार instance की जाँच करना।
Mutex / Lock	म्यूटेक्स / लॉक	<code>Swoole\Lock</code> एक म्यूटेक्स है जो एक समय में सिर्फ़ एक कॉलर को अंदर जाने देता है।
TTL (Time to Live)	TTL (जीवन-काल)	कैश एंट्री का TTL 60 सेकंड है, यानी 60 सेकंड बाद वह अपने-आप एक्सपायर हो जाएगी।
Dependency Injection	डिपेंडेंसी इंजेक्शन	Singleton के बजाय, <code>CacheManager</code> को कंस्ट्रक्टर के ज़रिए इंजेक्ट करना।
Single Responsibility Principle	सिंगल रिस्पॉन्सिबिलिटी प्रिंसिपल	Singleton इस प्रिंसिपल से टकराता है, क्योंकि यह दो काम एक साथ करता है।
Enum Singleton	एनम सिंगलटन	Java में, JVM एक enum-आधारित singleton के लिए इंस्टैंशिएशन गारंटी देता है।
Reflection	रिफ्लेक्शन	Java में <code>Constructor.newInstance()</code> से रिफ्लेक्शन के ज़रिए दूसरा instance बनाने की कोशिश।

General Words Glossary / सामान्य शब्दों की शब्दावली

English Word	Hindi Meaning / हिंदी अर्थ	Example / उदाहरण
Deceptively (small/simple)	धोखे से (छोटा/सरल) — दिखने में जितना लगे उतना नहीं	"The exam looked deceptively easy, but it took hours." परीक्षा दिखने में धोखे से आसान लगी, पर उसमें घंटों लग गए।
Interleave	आपस में मिल जाना, गड़मड़ होना	"The two conversations kept interleaving and became confusing." दोनों बातचीतें आपस में मिलती रहीं और उलझन भरी हो गई।
Coordinated	समन्वित, तालमेल वाला	"The dance troupe's movements were perfectly coordinated." डांस टोली की हरकतें एकदम तालमेल में थीं।
Threaded through (figurative)	पिरोया हुआ, गुज़ारा हुआ	"A theme of hope was threaded through the whole novel." पूरे उपन्यास में उम्मीद का एक भाव पिरोया हुआ था।
Sanctioned	स्वीकृत, अनुमोदित	"Only sanctioned vendors are allowed at the event." सिर्फ स्वीकृत विक्रेताओं को इस इवेंट में जगह मिलेगी।
Warm-up (figurative, of a system)	गरम होना, तैयार होना	"The server takes a minute to warm up after a restart." रीस्टार्ट के बाद सर्वर को तैयार होने में एक मिनट लगता है।
Sidesteps (verb)	टाल देना, बचकर निकल जाना	"Her answer sidestepped the real question." उसके जवाब ने असली सवाल को टाल दिया।
Genuinely	वाक़ई, सचमुच	"He was genuinely happy for his friend's success." वह अपने दोस्त की सफलता से वाक़ई खुश था।
Diverged (from each other)	अलग-अलग हो जाना, बिखर जाना	"Their opinions diverged after the meeting." मीटिंग के बाद उनकी राय अलग-अलग हो गई।
Out of habit	आदतन, आदत की वजह से	"He locked the door out of habit, even though no one else lived there." वह आदतन दरवाज़ा बंद कर देता, भले ही वहाँ कोई और न रहता हो।

This document follows the topic order of [algomaster.io](https://www.algomaster.io)'s Singleton lesson (a paid course) but is written independently — original English/Hindi explanations and original PHP code — rather than reproducing their (subscription-gated) material.

यह दस्तावेज़ [algomaster.io](https://www.algomaster.io) के Singleton पाठ (एक पेड कोर्स) के विषय-क्रम को फ़ॉलो करता है, लेकिन स्वतंत्र रूप से लिखा गया है — ओरिजिनल अंग्रेज़ी/हिंदी व्याख्याएँ और ओरिजिनल PHP कोड — बजाय इसके कि उनकी (सब्सक्रिप्शन-गेटेड) सामग्री को दोबारा प्रस्तुत किया जाए।